

Code Review Instruction

Version 1.0 — Languages: PHP, Swift, Python, C/C++ (ESP32), JavaScript

You are performing a **full code review** of a codebase and producing a quantified quality report. Work through every rule below, count findings against the defined denominator, compute the scores, and write a Markdown report.

Do not fix anything. This is an assessment run. Remediation happens in a separate pass driven by the report you produce.

1. Operating modes

Determine the mode before you start.

Mode A — Autonomous (Claude Code / agentic)

You have filesystem and shell access. Compute all denominators yourself.

- Use `cloc` for line counts if available; otherwise count logical lines yourself (see §3).
- Use `universal-ctags` for the function inventory if available; otherwise enumerate functions by reading the files.
- Write the report to `reports/code-review-YYYY-MM-DD.md`.
- Append one row per category to `reports/scores.csv`.
- Use available tooling, but **do not install anything**.

Mode B — Snapshot (chat / no filesystem)

You are given a `metrics.json` and a line-numbered source snapshot.

- **Take all mechanical denominators from `metrics.json`**. Do not recount them. Specifically: file count, LoC, function inventory, declaration count, encoding facts.
 - Derive only the judgment-based denominators yourself (error paths, entry points, resource acquisitions, critical paths, endpoints).
 - Emit the report as a Markdown document in the response.
 - If the snapshot is truncated or incomplete, say so explicitly in the report header and mark the result as **partial** — a partial run must not be presented as a comparable quality figure.
-

2. Scope

Review only the directory the user names. If none is named, review the repository root.

Excluded from review and from all counts:

```
vendor/  node_modules/  .git/  .history/  build/  dist/  out/
.pio/  Pods/  DerivedData/  __pycache__/  .venv/  venv/
```

```
*.min.js *.lock *-generated.* *_pb2.py
```

Also exclude third-party libraries vendored into the tree and machine-generated code. If you are unsure whether a directory is first-party, list it under *Assumptions* in the report header rather than silently deciding.

3. Counting rules

LoC means *logical lines of code*: physical lines minus blank lines, minus comment-only lines. Documentation headers therefore do **not** inflate LoC. This matters — several denominators are per-KLoC, and counting comments would let documentation improve the score by itself.

$KLoC = LoC / 1000$

Function means any named callable unit: function, method, procedure, constructor, initializer. Closures and lambdas are excluded unless they are assigned to a name at module or class level.

Declaration means any named function, type (class/struct/enum/protocol/ interface), constant, or module- or class-level variable.

4. Scoring model

Every rule is one of three types.

Type A — Ratio

Used when the rule has a natural countable unit.

$$S = (1 - F / N) * 100 \quad \text{clamped to } [0, 100]$$

F = number of violating instances, N = number of checkable units. If $N = 0$, the rule is **not applicable**: report *n/a* and drop its weight from the total.

Type B — Density

Used when there is no natural denominator.

$$d = F / KLoC$$

$$S = \max(0, (1 - d / d_{\max}) * 100)$$

$d_{\max}$ is the density at which the score reaches zero.

Type C — Gate

Binary and non-negotiable.

```
F = 0  -> PASS
F > 0  -> FAIL
```

Composite index

$$Q = \sum(w_i * S_i) / \sum(w_i) \quad \text{over all applicable rules}$$

Gate rule: if any Type C criterion fails, Q is capped at **50**, regardless of the computed value. State the cap explicitly in the report.

Rating: $Q \geq 85$ green · $70-84$ yellow · < 70 red.

Report Q rounded to one decimal place.

5. Rules

Each rule states what to check, what counts as one finding, and its denominator.

Group A — Structure

Rule 1 — Remove dead code

Type B · $d_{\max} = 5 / KLoC$ · Weight 2

Unused functions, variables, imports, constants, and unreachable branches. Commented-out code counts as dead code and is deleted — history lives in Git.

Exempt: deliberately retained debug switches that are documented as such.

One finding: one unused declaration, one unreachable branch, or one contiguous block of commented-out code.

Rule 2 — Reduce redundancy

Type B · $d_{\max} = 3 / KLoC$ · Weight 2

Extract from the third identical or near-identical occurrence onward. Two occurrences stay as they are if the cases could plausibly diverge on business grounds. Incidental similarity is not duplication.

One finding: one cluster of three or more duplicated occurrences (not one per occurrence).

Rule 3 — One responsibility per function

Type A · $N = \text{functions}$ · Weight 3

A function does one thing and is fully describable by its name. If the name needs an "and", split it.

Guide values — exceeding one is a finding unless the code justifies it:

- ≤ 50 lines
- ≤ 4 parameters (beyond that, pass a struct/object)
- nesting depth ≤ 3

One finding: one function violating one or more guide values without justification.

Rule 4 — Separate I/O from logic

Type A · N = functions · Weight 3

Computation, decision-making, and transformation belong in side-effect-free functions. Database access, HTTP, filesystem, hardware access, and output belong in their own functions.

Test: can the business logic be exercised without mocks?

One finding: one function that mixes business logic with I/O.

Group B — Readability

Rule 5 — Enforce naming conventions

Type A · N = declarations · Weight 2

Identifiers in English throughout.

Language	Functions / variables	Types	Constants
PHP (PSR-12)	<code>camelCase</code>	<code>PascalCase</code>	<code>UPPER_SNAKE</code>
Swift	<code>camelCase</code>	<code>PascalCase</code>	<code>camelCase</code>
Python (PEP 8)	<code>snake_case</code>	<code>PascalCase</code>	<code>UPPER_SNAKE</code>
C/C++	project convention	project convention	<code>UPPER_SNAKE</code>
JavaScript	<code>camelCase</code>	<code>PascalCase</code>	<code>UPPER_SNAKE</code>

Preprocessor macros are always `UPPER_SNAKE`.

Additional requirements:

- Booleans carry an `is` / `has` / `should` prefix.
- Units belong in the name: `timeoutMs`, `voltageMv`, `distanceCm`.
- No abbreviations except established ones (`id`, `url`, `api`, `http`).

One finding: one declaration violating any of the above.

Rule 6 — Document every function

Type A · N = functions · Weight 3

Every function, method, and procedure carries a header comment in the language's native format: PHPDoc, `///` in Swift, docstring (PEP 257) in Python, Doxygen in C/C++, JSDoc in JavaScript.

Minimum content: **one sentence stating the purpose**. No exceptions — getters and one-liners get their one sentence too.

Additionally, where applicable: every parameter with unit and value range, the return value, thrown errors, relevant side effects.

One finding: one function with a missing header, or with a header that omits an applicable element (parameters, return, errors).

Rule 7 — File header on every file

Type A · N = files · Weight 2

Every source file opens with a comment block containing:

- file name
- topic / purpose of the file, one to three sentences
- author
- creation date
- date of last change

Optional: project, license.

The change date must be maintained on every substantive edit. Recommend a pre-commit hook — hand-maintained dates rot.

One finding: one file with a missing header or a missing mandatory field.

Rule 8 — Comments explain *why*

Type B · d_max = 4 / KLoC · Weight 1

Inline comments justify decisions, workarounds, and non-obvious constraints. Comments that restate the code are deleted. `TODO` / `FIXME` only with a ticket reference — otherwise resolve or remove.

One finding: one comment that merely narrates the code, or one unreferenced `TODO` / `FIXME`.

Group C — Configuration

Rule 9 — Centralize configuration

Type B · d_max = 10 / KLoC · Weight 3

No magic numbers, no hard-coded paths, URLs, ports, timeouts, pin assignments, or thresholds in the code.

- Values that differ **per environment or per device** belong in a configuration file.
- Values that are **fixed by the domain** belong in a named constant at a defined place in the module — not in the config.

Exempt: 0, 1, -1, and array indices in obvious contexts.

One finding: one magic value or hard-coded environment-dependent value.

Rule 10 — Separate secrets from configuration

Type C · GATE

Credentials, API keys, and key material never appear in versioned files. Use a separate, `.gitignore`-excluded file or environment variables, plus a committed example file with placeholders.

Additionally check: has the key ever been committed? Check the Git history, not just the working tree. If yes, the report must state **rotate the key** — removing it from HEAD is not sufficient.

One finding: one secret in a versioned file or in history.

Group D — Robustness

Rule 11 — Handle errors, do not swallow them

Type A · N = error paths · Weight 4

N = number of calls that can signal an error: throwing calls, return values carrying an error indicator, `try` blocks.

No empty `catch` block, no ignored return value. Every error path leads to handling, to controlled propagation, or to a log entry with context. On failure the return value is defined — no implicit success.

One finding: one swallowed or unhandled error path.

Rule 12 — Validate input at the system boundary

Type A · N = entry points · Weight 4

N = number of functions that accept data from outside the process: HTTP requests, files, sensor readings, radio packets, IPC, user input.

Validate type, range, and plausibility at the boundary. Inside, trust validity rather than re-checking at every layer.

One finding: one entry point without adequate validation.

Rule 13 — Release resources

Type A · N = resource acquisitions · Weight 4

N = number of places where a resource is opened, allocated, or registered: file handles, DB connections, sockets, timers, observers, dynamic memory.

Release on **every** path, including error paths. Prefer language constructs (`with`, `RAII`, `defer`, `finally`) over manual release.

Language specifics:

- **Swift:** check for retain cycles — `[weak self]` in escaping closures.
- **C/C++ (ESP32):** check for heap fragmentation and blocking waits.

One finding: one acquisition without a guaranteed release, or one retain cycle.

Rule 14 — Security fundamentals

Type C (injection) + Type A (authorization) · N = endpoints · Weight 4

Gate portion — any finding fails the gate:

- Database access exclusively via prepared statements. String-concatenated SQL is a gate failure.
- Output escaped per context (HTML, attribute, JS, URL, shell).

Ratio portion — N = number of endpoints / externally reachable handlers:

- Every endpoint checks authentication **and** authorization — not merely the presence of a session.
- No sensitive data (passwords, tokens, personal data) in logs.

One finding: one injection-capable site (gate) or one endpoint with missing or incomplete authorization (ratio).

Group E — Supporting

Rule 15 — Unify logging

Type B · d_max = 5 / KLoC · Weight 1

One logging mechanism per project, with log levels. No stray `print`, `echo`, `console.log`, `NSLog`, or `Serial.print` on the production path.

One finding: one ad-hoc output call on a production path.

Rule 16 — Test critical paths

Type A · N = critical paths · Weight 3

N = number of business-logic functions, boundary conditions, and error cases that carry real risk. Do not count trivial code.

Every fixed bug has a test that would have reproduced it.

One finding: one critical path without coverage.

Rule 17 — Review dependencies

Type B (absolute) · d_max = 2 unused · Weight 1

$$S = \max(0, (1 - F / 2) * 100)$$

Remove unused dependencies, pin versions, and weigh benefit against maintenance cost for new ones.

One finding: one unused or unpinned dependency.

Rule 18 — Encoding and line endings

Type A · N = files · Weight 1

All text files UTF-8 without BOM, consistent line endings within the project. Where Windows systems are involved, also check path separators.

One finding: one file with a BOM, non-UTF-8 encoding, or deviating line endings.

6. Weights

#	Rule	Type	Denominator / d_max	Weight
1	Dead code	B	5 / KLoC	2
2	Redundancy	B	3 / KLoC	2
3	Single responsibility	A	functions	3
4	I/O separation	A	functions	3
5	Naming conventions	A	declarations	2
6	Function documentation	A	functions	3
7	File header	A	files	2
8	Why-comments	B	4 / KLoC	1
9	Configuration	B	10 / KLoC	3
10	Secrets	C	—	gate
11	Error handling	A	error paths	4
12	Input validation	A	entry points	4
13	Resource release	A	acquisitions	4
14	Security	C + A	endpoints	4
15	Logging	B	5 / KLoC	1
16	Critical-path tests	A	critical paths	3
17	Dependencies	B	2 (absolute)	1
18	Encoding	A	files	1

Σ weights = 43 (before deducting rules reported as **n/a**)

7. Report format

Write the report in **English**, as Markdown, in exactly this structure.

Code Review Report

****Project:**** <name>
****Scope:**** <reviewed path>
****Date:**** YYYY-MM-DD
****Mode:**** Autonomous | Snapshot
****Instruction version:**** 1.0
****Completeness:**** full | partial – <reason if partial>

1. Scope

Metric	Value
Files reviewed	
Logical LoC	
Functions	
Declarations	
Languages	
Excluded paths	

Assumptions

– <every judgment call made about scope or classification>

2. Score

#	Rule	Type	F	N	KLoC	Score	Weight	Weighted
1	Dead code	B	12	4.20	KLoC	42.9	2	85.8
...								

Gates

#	Rule	Result	Findings
10	Secrets	PASS / FAIL	
14	Injection	PASS / FAIL	

Composite index

...

$Q = \Sigma \text{ weighted} / \Sigma \text{ weights} = \text{<value>} / \text{<value>} = \text{<Q>}$

...

****Q = <value> – <green | yellow | red>****

<If capped: "Capped at 50.0 – gate failure in rule <n>. Uncapped value: <x>.">

3. Profile

Per-rule scores as a list, so the weak areas are visible without reading the findings. Name the three weakest rules explicitly and say what they mean in practice.

4. Findings

Grouped by rule, sorted by file, then by line.

Rule <n> – <title> (<F> findings)

File	Line	Category	Finding
src/api/User.php	142	Rule 6	`validateToken()` has no header comment.

The finding text is one sentence stating what is wrong or what should be done. List **all** findings – no truncation.

5. Remediation order

Ranked by weighted score impact against estimated effort. Name the three actions that raise `Q` the most per unit of work.

scores.csv (Mode A only)

Append one row per rule per run:

```
date,scope,rule,type,F,N,KLoC,score,weight
2026-08-06,hub/,1,B,12,,4.20,42.9,2
```

Plus one summary row with **rule=Q**.

8. Working rules

- **Report facts, not impressions.** Every finding has a file and a line number. A finding you cannot locate does not go in the report.
- **State uncertainty.** If you cannot decide whether something is a violation, list it under *Assumptions* and explain how you counted it.
- **Do not fix anything.** Assessment only.
- **Denominators must be reproducible.** In Mode B, use the numbers from **metrics.json** verbatim. If your own reading contradicts them, note the discrepancy in the report instead of silently overriding.
- **Do not weight by severity within a rule.** One finding is one finding; the weighting happens at rule level.